

Zero-knowledge forensic attestation

The verdict is visible. The victim is not.

6

ELEMENTS BOUND
IN THE
COMMITMENT

5

DETECTORS, EXACT
RATIONALS

4

RED TEAM ROUNDS
ON OURSELVES

27/21

FINDINGS RAISED /
FIXED

11

ATTACKS EXECUTED
& DEFEATED

0

FLOATS IN THE
DECISION PATH

VELO is not a demo that renders a verdict. It is a system with **provable properties** — determinism, tamper evidence, and a legal admissibility rule enforced as a cryptographic constraint rather than a promise — with a contract deployed to Midnight's live [preview](#) network. **The properties are the product.**

WHAT THE CIRCUIT ACTUALLY BINDS

- **Six elements inside one hash** — domain, analysis fingerprint, custody tip, verdict, corroboration count, salt. An earlier version hashed three and left the verdict a free public argument: a sealed **NOISE** analysis could have been attested as **MALICE**. We found it ourselves (F3).
- **The admissibility rule is a circuit constraint.** **MALICE** without two independent sources does not give a rejected transaction — it fails to produce a proof at all. No code path, no admin override.
- **Replay is blocked** — re-attesting the same tuple used to inflate the counter, manufacturing fake corroboration for the price of a second fee (G2). One **assert** closed it.
- **The dual-ledger boundary is compiler-enforced.** Four private witnesses enter; none leaves. Crossing without **disclose()** is a compile error, not a code-review convention.

ENGINEERING RIGOR

- **Determinism is enforced.** Exact rational arithmetic over **bigint**; comparison cross-multiplies and never divides, so no rounding step exists between evidence and verdict. Thresholds **33/100** and **8/100**.
- **Fail-closed in four places.** No custody → **ABSTAIN** before any detector is read. Unargued malice degrades to **SUSPICION**. Missing secrets throw. Malformed input yields a sentence, not a stack trace.
- **Built so an opposing expert's reimplement agrees.** Keys sort by Unicode code point, not UTF-16 unit; unsafe numbers throw instead of coercing; duplicate JSON keys are rejected. Two independent canonicalizers pinned to 20 shared vectors.
- **The model is out of the decision path.** No LLM scores, classifies or decides. The verdict is sealed before any narration.

THE FIVE THINGS YOU CANNOT FAKE IN TEN HOURS

- **A test written to prove a limitation exists.** **pipeline.test.ts** asserts that custody truncation passes local verification. Nobody writes that test to look good.
- **A mitigation that failed, documented as having failed.** The seed-redaction wrapper passed 10/10 under Node and the seed printed in full on the first real deploy — the deploy runs under Bun, which bypasses **process.stdout.write**.
- **A test that passed while the thing it tested was visibly broken.** It observed through the exact mechanism Bun bypasses, so the capture stayed empty and the assertion passed. **A check that observes nothing passes everything.** It now greps the bytes a child process actually emitted.
- **A falsified attack kept in the record.** F15's prompt injection did not work. We wrote down that it did not work, and that this does not close the question.
- **A corpus checked against its own engine.** 8 of 13 demo cases diverged from their documented verdict — including one we downgraded from **MALICE** because the real gate would not pass it.

Demos accumulate features. Engineered systems accumulate negative results. Ours are in the repository, dated — alongside three Midnight platform walls (NIGHT is not DUST; error 170 is a fee proof, not your contract) that we documented rather than quietly worked around.

WHAT THIS DOES NOT ESTABLISH — STATED BY US, BEFORE YOU ASK

G1 — witness provenance. A ZK circuit proves a relationship between the witness values it is *given*, not that those values describe anything that happened. A prover bypassing the normal calling code can produce a valid proof for evidence that was never analyzed. Fix on the roadmap: an accredited-examiner credential.

G3 — independence is declared, not proven. The circuit checks the count, not that the sources are independent. Two files from one physical disk can yield two. **This survives even if G1 is solved.**

End-to-end. Prover and verifier keys are generated and the contract is deployed, but a proof has not yet been exercised against a live test vector. And VELO removes post-hoc tampering with a sealed verdict — it does not remove a corrupt examiner at the moment of analysis.

Atestación forense de conocimiento cero

El veredicto se ve, la víctima no.

6 ELEMENTOS ATADOS EN EL COMMITMENT	5 DETECTORES, RACIONALES EXACTOS	4 RONDAS DE RED TEAM PROPIAS	27/21 HALLAZGOS / ARREGLADOS	11 ATAQUES CORRIDOS Y BLOQUEADOS	0 FLOATS EN LA DECISIÓN
---	--	--	--	--	---

VELO no es un demo que dibuja un veredicto. Es un sistema con **propiedades demostrables** — determinismo, evidencia de manipulación, y una regla de admisibilidad legal impuesta como restricción criptográfica en vez de como promesa — con un contrato desplegado en la red `preview` real de Midnight. **Las propiedades son el producto.**

QUÉ ATA REALMENTE EL CIRCUITO

- **Seis elementos adentro de un hash** — dominio, fingerprint del análisis, tip de custodia, veredicto, conteo de corroboración, salt. Una versión anterior hasheaba tres y dejaba el veredicto como argumento público libre: un análisis sellado como `NOISE` podía atestarse como `MALICE`. Lo encontramos nosotros (F3).
- **La regla de admisibilidad es una restricción del circuito.** `MALICE` sin dos fuentes independientes no da una transacción rechazada: directamente no produce prueba. Sin camino de código, sin override.
- **El replay está bloqueado** — re-atestar la misma tupla inflaba el contador y fabricaba corroboración falsa por el precio de una segunda fee (G2). Un `assert` lo cerró.
- **El límite de dual-ledger lo impone el compilador.** Entran cuatro witnesses privados; no sale ninguno. Cruzar sin `disclose()` es un error de compilación, no una convención de code review.

RIGOR DE INGENIERÍA

- **El determinismo se impone.** Aritmética racional exacta sobre `bigint`; la comparación multiplica cruzado y nunca divide, así que no hay redondeo entre la evidencia y el veredicto. Umbrales `33/100` y `8/100`.
- **Fail-closed en cuatro lugares.** Sin custodia → `ABSTAIN` antes de leer un detector. La malicia sin discutir baja a `SUSPICION`. Los secretos faltantes tiran error. La entrada malformada devuelve una oración, no un stack trace.
- **Hecho para que la reimplementación de la contraparte coincida.** Claves por code point Unicode y no por unidad UTF-16; los números inseguros tiran error; las claves JSON duplicadas se rechazan. Dos canonicalizadores fijados a 20 vectores compartidos.
- **El modelo está fuera del camino de decisión.** Ningún LLM puntúa, clasifica ni decide. El veredicto queda sellado antes de cualquier narración.

LAS CINCO COSAS QUE NO SE PUEDEN FINGIR EN DIEZ HORAS

- **Un test escrito para probar que existe una limitación.** `pipeline.test.ts` afirma que el truncamiento de custodia pasa la verificación local. Nadie escribe ese test para quedar bien.
- **Una mitigación que falló, documentada como fallida.** El wrapper de redacción del seed pasó 10/10 bajo Node y el seed se imprimió entero en el primer deploy real: el deploy corre bajo Bun, que se saltea `process.stdout.write`.
- **Un test que pasó mientras lo que testeaba estaba visiblemente roto.** Observaba por el mecanismo exacto que Bun se saltea: la captura quedó vacía y la aserción pasó. **Un check que no observa nada aprueba todo.** Hoy busca en los bytes que un subproceso realmente emitió.
- **Un ataque falsado conservado en el registro.** La prompt injection de F15 no funcionó. Escribimos que no funcionó, y que eso no cierra la pregunta.
- **Un corpus chequeado contra su propio motor.** 8 de 13 casos de demo discrepaban con su veredicto documentado — incluido uno que bajamos de `MALICE` porque el gate real no lo dejaba pasar.

Los demos acumulan features. Los sistemas de ingeniería acumulan resultados negativos. Los nuestros están en el repositorio, fechados — junto a tres paredes de la plataforma Midnight (NIGHT no es DUST; el `error 170` es la prueba de la fee, no tu contrato) que documentamos en vez de esquivar en silencio.

QUÉ NO ESTABLECE ESTO — LO DECIMOS NOSOTROS, ANTES DE QUE PREGUNTEN

G1 — procedencia del witness. Un circuito ZK prueba una relación entre los valores de witness que *recibe*, no que esos valores describan algo que ocurrió. Un prover que evita el código normal puede producir una prueba válida sobre evidencia que nunca se analizó. Arreglo en el roadmap: una credencial de perito acreditado.

G3 — la independencia se declara, no se prueba. El circuito chequea el conteo, no que las fuentes sean independientes. Dos archivos de un mismo disco físico pueden dar dos. **Sobrevive incluso si G1 se resuelve.**

End-to-end. Las claves de prover y verifier están generadas y el contrato desplegado, pero todavía no se ejercitó una prueba contra un vector de test vivo. Y VELO elimina la manipulación posterior de un veredicto sellado — no elimina a un perito corrupto en el momento del análisis.